

CCBB 企业级多智能体协作系统 (Multi-Agent System)架构说明书

系统概述与架构设计 (Page 1: Overview & Architecture)

版本: V1.0 日期: 2026-03-15

1. 系统背景与目标

随着企业业务复杂度的提升，单一的大语言模型（LLM）已无法满足跨部门、多步骤的复杂任务处理需求。本项目旨在设计一个多智能体协作网络（Multi-Agent Collaboration Network），通过引入多个具备特定领域知识的 Agent，实现任务的自动路由、规划与并行处理。系统的核心目标是将客户工单的平均处理时间（AHR）缩短 40%。

2. 核心系统架构图

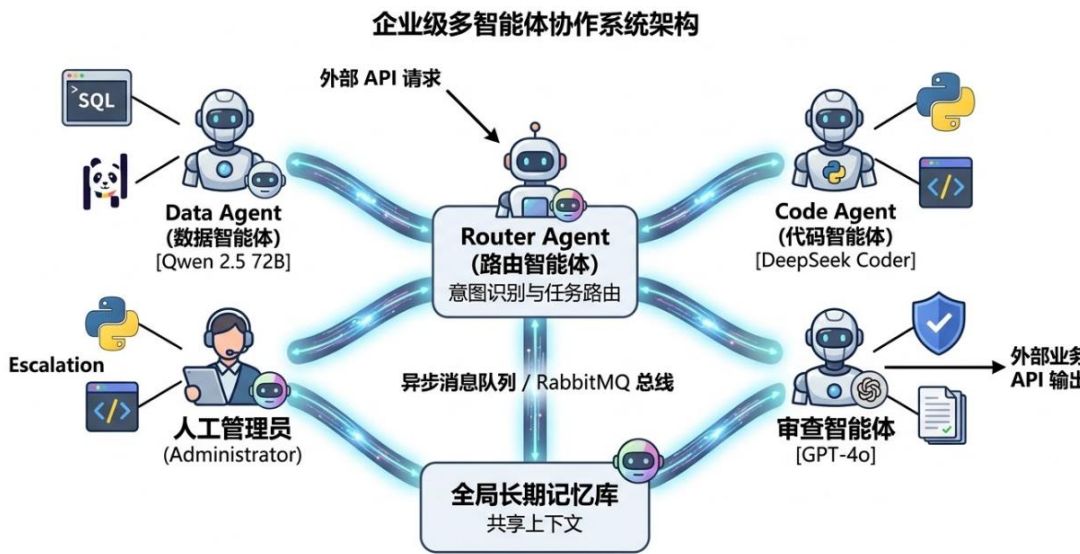


图 1: 多智能体协作网络拓扑图

如上图所示，系统采用中心辐射型（Hub-and-Spoke）架构。Router Agent 位于中心，负责接收外部 API 的高并发请求，并根据意图分类模型的结果，将任务分发给下游的专业 Agent 集群。所有 Agent 共享一个全局的长期记忆库（Long-term Memory Bank），以保证上下文的一致性。

智能体角色与规格说明 (Page 2: Agent Roles & Specs)

(这页用来测试系统把 PDF 里的表格切分并向量化的能力, 这对很多 RAG 系统都是个挑战。)

3. 核心智能体规格矩阵

为了实现功能解耦, 系统当前版本部署了以下四个核心智能体。每个智能体都配置了不同的底层模型和独立 Prompt。

智能体名称 (Agent)	底 层 模 型 (LLM)	核心职责 (Core Function)	依 赖 工 具 (Tools / APIs)
Router Agent	Gemini 2.0 Flash	意图识别、任务分发、权限校验	API Gateway, Auth Token
Data Agent	Qwen 2.5 72B	数据库查询、报表生成、数据清洗	SQL Client, Pandas
Code Agent	DeepSeek Coder	代码编写、Bug 修复、单元测试	Python REPL, GitHub API
Review Agent	GPT-4o	质量把控、合规审查、最终输出	Linter, Policy Database

4. 通信协议

Agent 之间采用异步消息队列 (基于 RabbitMQ) 进行通信。每个 Agent 在完成自己的子任务后, 会将结果打包成标准的 JSON 格式发送到消息总线。如果遇到无法解决的歧义, Agent 会触发 Escalation 事件, 将控制权交还给人类管理员。

执行 workflow 演示 (Page 3: Workflow Execution)

5. 典型任务执行链路

以下是一个处理用户复杂请求的完整生命周期示例。

用户输入请求：“请帮我查询上个月华东区的销售数据，并写一段 Python 代码把销量前三的城市画成柱状图。”

执行步骤：

1. **任务接收与解析：**Router Agent 接收到请求，将其拆解为两个子任务：任务 A（数据查询）和任务 B（代码编写）。
2. **数据处理 (并行)：**Data Agent 接收任务 A，连接到企业后台 MySQL 数据库，执行 SQL 查询，提取出华东区包含城市和销量的 CSV 数据流。
3. **代码生成 (并行)：**Code Agent 接收任务 B 以及 Data Agent 传来的数据流结构，编写出基于 matplotlib 的柱状图绘制脚本。
4. **合并与审查：**Review Agent 对生成的代码进行安全性扫描（防止 SQL 注入或恶意操作），确认无误后，将最终的数据和代码一并返回给用户端。

6. 总结与未来规划

当前 V1.0 版本已成功验证了静态规划下的多 Agent 协作能力。未来 V2.0 计划引入强化学习 (RLHF)，使 Agent 集群具备动态自我纠错和自主反思的能力。